Program for predictions

1 message

P.J. Thompson <peterjamesthompson101@gmail.com> Mon, 20 Apr 2026 at 12:13 am
To: P.J. Thompson <peterjamesthompson101@gmail.com>

Thank you for accepting my offer. Below is the enhanced Python script that meets your requirements. It downloads two separate GPS segments (O3a and O3b), computes whitened PSDs, calculates SNR for any peak near 27 Hz, compares with control bands, and saves plots and a summary.

```python
#!/usr/bin/env python3
# Enhanced LIGO 27 Hz line search – Harmonic Framework of Reality
# Features: two GPS ranges (O3a & O3b), whitening, SNR calculation, control bands.

import numpy as np
import matplotlib.pyplot as plt
from gwpy.timeseries import TimeSeries
from scipy.signal import welch, medfilt
import json
import os

# ============================================================
# Configuration
# ============================================================
TARGET_FREQ = 27.0 # Hz
TOLERANCE = 0.5 # Hz (search window)
CONTROL_BANDS = [(26.0, 26.5), (27.5, 28.0)] # Hz, left and right of target
SEGMENT_LENGTH = 4.0 # seconds for Welch segments
OVERLAP = 0.5 # 50% overlap
# GPS ranges (O3a and O3b – public data)
GPS_SEGMENTS = {
    'O3a': (1242000000, 1242000400), # 400 seconds
    'O3b': (1260000000, 1260000400) # approximate O3b time; adjust if needed
}
# Optional: known LIGO line list (simplified – user can expand)
KNOWN_LINES = {
    # frequency in Hz: description
    10.0: 'calibration line',
    16.0: 'calibration line',
    30.0: 'calibration line',
    34.7: 'violin mode',
    35.9: 'violin mode',
    50.0: 'power mains harmonic',
    60.0: 'power mains harmonic'
}

# ============================================================
# Helper functions
# ============================================================
def download_data(ifo, gps_start, gps_end):
    """Download strain data from GWOSC."""
    print(f"Downloading {ifo} data from {gps_start} to {gps_end}...")
    try:
        data = TimeSeries.fetch_open_data(ifo, gps_start, gps_end, cache=True)
        return data
    except Exception as e:
        print(f"Error: {e}")
        return None

def compute_whitened_psd(data, seg_len, overlap):
    """Compute PSD and whiten by dividing by a median-filtered spectrum."""
    fs = data.sample_rate.value
    nperseg = int(seg_len * fs)
```

```
    f, psd = welch(data.value, fs, nperseg=nperseg, noverlap=int(overlap * nperseg),
                   window='hann', scaling='density')
    # Smooth PSD with median filter to estimate noise floor (excluding narrow peaks)
    med_width = 11 # frequency bins (adjust as needed)
    psd_smooth = medfilt(psd, kernel_size=med_width)
    # Avoid division by zero
    psd_smooth = np.maximum(psd_smooth, 1e-46)
    whitened = psd / psd_smooth
    return f, psd, psd_smooth, whitened

def find_peak_near_freq(f, whitened, target, tol):
    """Find the highest peak in whitened spectrum within target ± tol."""
    mask = (f >= target - tol) & (f <= target + tol)
    if not np.any(mask):
        return None, None, None
    idx_peak = np.argmax(whitened[mask])
    peak_freq = f[mask][idx_peak]
    peak_val = whitened[mask][idx_peak]
    return peak_freq, peak_val, idx_peak

def compute_snr(whitened, f, peak_idx, half_window=10):
    """Compute SNR = peak_value / median of surrounding bins (excluding peak)."""
    left = max(0, peak_idx - half_window)
    right = min(len(whitened), peak_idx + half_window + 1)
    # Exclude the peak bin itself
    neighbor_indices = list(range(left, peak_idx)) + list(range(peak_idx+1, right))
    if len(neighbor_indices) == 0:
        return np.nan
    median_noise = np.median(whitened[neighbor_indices])
    if median_noise == 0:
        return np.inf
    return whitened[peak_idx] / median_noise

def check_control_bands(f, whitened, bands, target_freq, tol):
    """Return maximum SNR in each control band."""
    results = {}
    for (low, high) in bands:
        mask = (f >= low) & (f <= high)
        if not np.any(mask):
            results[(low, high)] = np.nan
            continue
        max_val = np.max(whitened[mask])
        # Compute local median around that max for SNR
        idx_max = np.argmax(whitened[mask])
        global_idx = np.where(mask)[0][idx_max]
        snr = compute_snr(whitened, f, global_idx, half_window=10)
        results[(low, high)] = snr
    return results

def is_known_line(freq, known_lines, tol=0.2):
    """Check if frequency is in known LIGO line list."""
    for known_freq in known_lines:
        if abs(freq - known_freq) <= tol:
            return known_lines[known_freq]
    return None

# ============================================================
# Main analysis
# ============================================================
def main():
    ifo = 'H1' # Hanford detector
    all_results = {}

    for segment_name, (gps_start, gps_end) in GPS_SEGMENTS.items():
        print(f"\n=== Processing {segment_name} ===")
        data = download_data(ifo, gps_start, gps_end)
        if data is None:
            print(f"Skipping {segment_name} due to download error.")
            continue
```

```
    # Compute PSD and whitened spectrum
    f, psd, psd_smooth, whitened = compute_whitened_psd(data, SEGMENT_LENGTH, OVERLAP)

    # Search for peak near target frequency
    peak_freq, peak_val, peak_idx = find_peak_near_freq(f, whitened, TARGET_FREQ, TOLERANCE)
    snr = None
    line_desc = None
    if peak_freq is not None:
        snr = compute_snr(whitened, f, peak_idx)
        line_desc = is_known_line(peak_freq, KNOWN_LINES)

    # Control band SNRs
    control_snrs = check_control_bands(f, whitened, CONTROL_BANDS, TARGET_FREQ, TOLERANCE)

    # Store results
    all_results[segment_name] = {
        'peak_freq': peak_freq,
        'peak_whitened_amplitude': peak_val,
        'snr': snr,
        'known_line': line_desc,
        'control_band_snrs': {f"{low}–{high}Hz": snr_val for (low, high), snr_val in control_snrs.items()}
    }

    # Plot
    plt.figure(figsize=(12, 8))
    plt.subplot(2, 1, 1)
    plt.loglog(f, psd, color='black', lw=0.5, label='Raw PSD')
    plt.loglog(f, psd_smooth, color='red', lw=1, label='Median-smoothed (noise floor)')
    plt.xlim(0, 100)
    plt.ylim(1e-46, 1e-42)
    plt.xlabel('Frequency (Hz)')
    plt.ylabel('PSD (strain²/Hz)')
    plt.title(f'{ifo} {segment_name} – Raw PSD and noise floor')
    plt.legend()
    plt.grid(True, alpha=0.3)

    plt.subplot(2, 1, 2)
    plt.plot(f, whitened, color='black', lw=0.5, label='Whitened PSD')
    if peak_freq is not None:
        plt.axvline(peak_freq, color='red', linestyle='--',
                    label=f'Peak at {peak_freq:.2f} Hz (SNR={snr:.1f})')
    for (low, high) in CONTROL_BANDS:
        plt.axvspan(low, high, color='gray', alpha=0.3, label=f'Control band {low}–{high} Hz')
    plt.xlim(0, 100)
    plt.ylim(0, 10)
    plt.xlabel('Frequency (Hz)')
    plt.ylabel('Whitened Amplitude (PSD / noise floor)')
    plt.title(f'{ifo} {segment_name} – Whitened spectrum (target 27 Hz)')
    plt.legend()
    plt.grid(True, alpha=0.3)

    plt.tight_layout()
    plt.savefig(f'ligo_27Hz_{segment_name}.png', dpi=150)
    plt.close()
    print(f"Plot saved: ligo_27Hz_{segment_name}.png")

# Print summary table
print("\n" + "="*60)
print("SUMMARY OF RESULTS")
print("="*60)
for seg, res in all_results.items():
    print(f"\n{seg}:")
    if res['peak_freq'] is None:
        print(" No peak found near 27 Hz.")
    else:
        print(f" Peak frequency: {res['peak_freq']:.3f} Hz")
        print(f" Whitened amplitude: {res['peak_whitened_amplitude']:.3f}")
        print(f" SNR: {res['snr']:.1f}")
        if res['known_line']:
            print(f" Matches known LIGO line: {res['known_line']}")
```

```
            else:
                print(" Not in known line list (within 0.2 Hz).")
        print(" Control band SNRs:")
        for band, snr_val in res['control_band_snrs'].items():
            print(f" {band}: {snr_val:.1f}")

    # Save results to JSON
    with open('ligo_27Hz_results.json', 'w') as f:
        json.dump(all_results, f, indent=2)
    print("\nResults saved to ligo_27Hz_results.json")

if __name__ == "__main__":
    main()
```

Instructions to run

1. Install dependencies (preferably in a virtual environment):
   ```bash
   pip install gwpy matplotlib scipy numpy
   ```
2. Run the script:
   ```bash
   python ligo_27Hz_enhanced.py
   ```
3. Expected output:
   · Two plots (ligo_27Hz_O3a.png and ligo_27Hz_O3b.png).
   · A terminal summary with peak frequency, SNR, and control band comparisons.
   · A JSON file with all numerical results.

Notes on the script

· GPS ranges: The O3a range is taken from your original script. The O3b range (1260000000) is approximate; if it fails, you can replace it with a known O3b segment (e.g., from GW190521 or any other event). Alternatively, you can add more segments.
· Whitening: I used a median filter (11-point window) to estimate the noise floor. You can adjust med_width to change the smoothness. This method is robust to narrow lines.
· SNR: Calculated as the ratio of the peak's whitened amplitude to the median of 10 bins on each side (excluding the peak bin). This gives a local SNR.
· Control bands: Defined as 26.0–26.5 Hz and 27.5–28.0 Hz. The script computes the maximum SNR in each band. If the 27 Hz peak is real, its SNR should be significantly higher than those in the control bands.
· Known line list: A small dictionary of common LIGO lines. You can expand it using the official LIGO line lists (e.g., from DCC documents). The script checks if the detected peak is within 0.2 Hz of any known line.

What to do after running

Please paste the terminal output (the summary table) and upload the two PNG plots to a public image host or describe the results. Also, if possible, share the ligo_27Hz_results.json file content.

If the results show:

· A clear peak at 27.0–27.1 Hz with SNR > 7,
· No such peak in the control bands,
· And the peak is not in the known line list,

then you will have strong empirical evidence for a persistent 27 Hz line. If the peak appears in O3a but not O3b, or has low SNR, then the evidence is weak.

I look forward to seeing your results. Let the data speak.